PES UNIVERSITY • CSE SEM 5 DBMS LEVEL 3 PROJECT

Financial Market Anomaly & Crash Detection System

Project Architecture, Equal Work Division Matrix & Git Development Roadmap

Course: DBMS Experiential Learning
Faculty Guide: Dr. Nagasundari S
Section: A | **Team Size:** 2 Members
Stack: PostgreSQL 17 + pgvector + Next.js

Member 1: Amrit Raj Anmol
SRN: **PES2UG24CS056** | Section A

Completed: 18-Table Relational DDL & HNSW Indexes.
Focus: Semantic Vector Search (pgvector HNSW), Watchlists & User Preference System, Automated Alert Triggers, Role-based Auth (JWT).

Member 2: Aman Raj
SRN: **PES2UG24CS053** | Section A

Current Step: Implementation Roadmap & Work Division.
Focus: Market OHLCV Ingestion Pipeline, Stored Procedure Anomaly Engine, Analytical Views, Next.js Dashboard & Charts UI.

1. EXECUTIVE SUMMARY & SYLLABUS ALIGNMENT

Problem Statement: Financial markets generate vast amounts of structured price/volume series alongside unstructured news streams. Conventional dashboards only display raw prices without correlating sudden price collapses, volatility spikes, and news sentiment. This system detects equity anomalies in real time, correlates them with historical market events, and enables semantic search over financial news using vector embeddings.

Level 3 Rubric Compliance: Per syllabus guidelines, "*Projects should be completed as 2 member teams. Each team member must have clearly defined responsibilities and contribute to both database and application development. The project should go beyond basic CRUD.*" Both members have an exact 50% split across database logic, Next.js routes, UI components, and evaluation requirements.

2. 50/50 EQUAL WORK DIVISION MATRIX

Project Layer	Aman Raj (PES2UG24CS053)	Amrit Raj Anmol (PES2UG24CS056)
Data Pipeline & Synthetics	• `scripts/seed_market_data.py` Generates 8 sectors, 20 stocks (NASDAQ/NSE), 12 months of daily OHLCV prices, and injects synthetic crash/spike scenarios.	• `scripts/seed_news_embeddings.py` Scrapes/generates 50+ financial news headlines, computes 384-d dense embeddings via `all-MiniLM-L6-v2`, and links to events.
Database Logic (Advanced SQL)	• **Stored Procedure:** `sp_detect_market_anomalies` (Window functions: rolling 20-day Z-score & volume surge). • **Views:** `vw_stock_daily_stats` & `vw_market_overview`. • **Complex DML:** Multi-table JOINs, GROUP BY, HAVING, Subqueries & ACID transactions with ROLLBACK safety.	• **Already Executed:** 18-table DDL, constraints, B-Tree and HNSW vector cosine indexing (`embedding vector_cosine_ops`). • **Stored Function:** `fn_match_news_semantic` (Cosine distance `<=>` with dynamic threshold). • **Trigger:** `trg_on_anomaly_create_alert` (Auto-alerts watched stocks on anomaly insertion).
Backend APIs (Next.js Routes)	• Database connection pool (`lib/db.ts`). • `/api/stocks` & `/api/stocks/[ticker]` (Candlestick data). • `/api/anomalies` (Query anomalies, execute detection). • `/api/analytics/summary` (Market breadth & turnover).	• Auth session & password hashing (`lib/auth.ts`). • `/api/auth/[login|register|me]` (RBAC: Admin / Analyst). • `/api/semantic-search` (Text-to-vector query engine). • `/api/watchlist` & `/api/alerts` (Full CRUD).
Frontend Interface (Next.js)	• Executive Market Dashboard (`app/page.tsx`). • Interactive Stock Charts with anomaly event markers (`StockChart.tsx` using Recharts). • Anomaly Audit Explorer with severity filters & indicator breakdown modal (`AnomalyTable.tsx`).	• Global Navigation Bar & Alert Bell (`AlertDropdown.tsx`). • Semantic Vector Search page with similarity confidence cards (`app/semantic-search/page.tsx`). • Watchlist Manager UI (`app/watchlist/page.tsx`). • Authentication pages (Login / Register).
Documentation & Viva Defense	• `docs/DATA_DICTIONARY.md` • `docs/SYSTEM_ARCHITECTURE.md` • Defense: Statistical Anomaly Math & Window Functions.	• `README.md` (Step-by-step reproduction guide) • `docs/ADVANCED_DATABASE_FEATURES.md` • Defense: pgvector HNSW Indexing & Event Triggers.

3. COMPLETE PROJECT ARCHITECTURE & FILE STRUCTURE

```
DBMS-Mini-Project/
├── .env.example                          # Sample DB credentials (HOST, PORT, USER, PWD, JWT_SECRET)
├── .gitignore                            # Ignore node_modules, .env.local, .next, __pycache__
├── README.md                             # Setup & Execution Documentation [Amrit]
├── package.json                          # Next.js 14/15, Tailwind, Recharts, pg, bcryptjs, jose
├── database/
│   ├── ddl/
│   │   └── financial_market_schema.sql   # 18-Table DDL & HNSW Indexes [Amrit - Done]
│   └── advanced/
│       ├── 01_procedures_functions.sql   # sp_detect_market_anomalies [Aman] & fn_match_news_semantic [Amrit]
│       ├── 02_triggers.sql               # trg_on_anomaly_create_alert trigger [Amrit]
│       ├── 03_views.sql                  # vw_stock_daily_stats & vw_market_overview [Aman]
│       └── 04_evaluation_queries.sql     # Grading queries: Group By, Having, Subqueries, ACID TX [Aman]
├── scripts/
│   ├── requirements.txt                  # psycopg2-binary, sentence-transformers, pandas, numpy
│   ├── seed_market_data.py               # Synthetic OHLCV & crash data generator [Aman]
│   └── seed_news_embeddings.py           # News corpus & 384d vector embedding generator [Amrit]
├── src/
│   ├── lib/
│   │   ├── db.ts                         # PostgreSQL connection pool using pg [Aman]
│   │   ├── auth.ts                       # Password hashing & JWT verification [Amrit]
│   │   └── embeddings.ts                 # MiniLM embedding pipeline helper [Amrit]
│   ├── app/
│   │   ├── layout.tsx & globals.css      # Global layout & styling [Shared]
│   │   ├── page.tsx                      # Main Market Analytics Dashboard [Aman]
│   │   ├── login/page.tsx & register/page.tsx # Authentication pages [Amrit]
│   │   ├── stocks/[ticker]/page.tsx      # Stock OHLCV chart & metrics [Aman]
│   │   ├── anomalies/page.tsx            # Anomaly audit table & filters [Aman]
│   │   ├── semantic-search/page.tsx      # pgvector semantic search page [Amrit]
│   │   ├── watchlist/page.tsx            # User watchlist & alert rules [Amrit]
│   │   └── api/
│   │       ├── auth/[login|register|me]/ # Auth API endpoints [Amrit]
│   │       ├── stocks/ & stocks/[ticker]/ # Market data endpoints [Aman]
│   │       ├── anomalies/ & anomalies/[id]/ # Anomaly query & execution endpoints [Aman]
│   │       ├── semantic-search/          # Vector query endpoint [Amrit]
│   │       └── watchlist/ & alerts/      # Watchlist and Alert CRUD endpoints [Amrit]
│   └── components/
│       ├── Navbar.tsx & AlertDropdown.tsx # Navigation & notification popup [Amrit]
│       ├── MarketOverviewCard.tsx        # Sector performance & anomaly counters [Aman]
│       ├── StockChart.tsx                # Interactive Candlestick / Line chart [Aman]
│       ├── AnomalyTable.tsx              # Anomaly logs with severity badges [Aman]
│       ├── SemanticSearchBox.tsx         # Vector search input & score cards [Amrit]
│       └── WatchlistManager.tsx          # Watchlist stock manager [Amrit]
└── docs/
    ├── DATA_DICTIONARY.md                # Schema tables, columns & constraints description [Aman]
    ├── SYSTEM_ARCHITECTURE.md            # System data flow & ER diagram references [Aman]
    └── ADVANCED_DATABASE_FEATURES.md     # Technical explanation of 5 advanced DB concepts [Amrit]
```

4. DETAILED TECHNICAL SPECIFICATIONS OF KEY PROGRAMS

A. Market Data Pipeline & Anomaly Engine (Aman Raj - PES2UG24CS053)

- **`scripts/seed_market_data.py`:** Populates reference tables (`sector`, `company`, `exchange`, `stock`, `trading_day`). Generates 1 year of daily OHLCV records for 20 equities. Injects synthetic anomalies: *flash crashes* (sharp 8–15% drops on 4x average volume), *speculative spikes*, and *volatility divergence*.
- **`database/advanced/01_procedures_functions.sql` (Anomaly SP):** Defines `sp_detect_market_anomalies(p_trade_date DATE, p_std_dev_threshold NUMERIC)`. Uses PostgreSQL analytical window functions: `AVG(close_price) OVER (PARTITION BY stock_id ORDER BY trade_date ROWS BETWEEN 20 PRECEDING AND 1 PRECEDING)`. Computes rolling Z-scores for price and volume; if deviations surpass the threshold, inserts rows into `anomaly` and records indicator metrics into `anomaly_indicator_value`.
- **`database/advanced/03_views.sql`:** `vw_stock_daily_stats` (calculates 20-day SMA, daily return %, intraday spread) and `vw_market_overview` (sector-level market capitalization turnover and open anomaly counts).
- **`database/advanced/04_evaluation_queries.sql`:** Fully satisfies Level 3 evaluation queries: JOINs across 4 tables, GROUP BY sector with HAVING clause filtering for average anomaly scores, correlated subqueries identifying decoupling stocks, and ACID transaction blocks with ROLLBACK safety.

B. Semantic Vector Search, Triggers & Auth (Amrit Raj Anmol - PES2UG24CS056)

- **`scripts/seed_news_embeddings.py`:** Seeds realistic market articles (macro rate hikes, bank run liquidity scares, tech earnings beats). Encodes text into **384-dimensional dense vectors** using the pretrained `all-MiniLM-L6-v2` model. Stores vectors in `news_article.embedding` and links articles to `market_event`.
- **`database/advanced/01_procedures_functions.sql` (Vector SP):** Defines `fn_match_news_semantic(p_query_vector VECTOR(384), p_threshold NUMERIC, p_limit INT)`. Leverages the **HNSW cosine index (`embedding vector_cosine_ops`)** using the cosine distance operator `news_article.embedding <=> p_query_vector` to perform sub-millisecond similarity matching.
- **`database/advanced/02_triggers.sql`:** Defines `trg_on_anomaly_create_alert` (an `AFTER INSERT ON anomaly` trigger). Whenever an anomaly is logged, it scans `watchlist_item` for users monitoring that stock and automatically creates personalized unread entries in `alert`.
- **`src/lib/auth.ts` & API:** Implements secure password hashing with `bcryptjs`, generates signed JWT session tokens, and enforces role-based access control (differentiating `Admin` vs. `Analyst`).

5. STEP-BY-STEP GITHUB COMMIT ROADMAP (PART 1: COMMITS 1 TO 14)

The commit sequence below guarantees balanced, parallel collaboration. Alternate commits between both authors as outlined.

Sprint 0 & 1: Database Foundation & Advanced Database Logic

Commit 1: feat(db): initialize database schema and ddl for financial market system — AMRIT RAJ ANMOL • Baseline

Creates 18 normalized tables with foreign keys, checks, B-tree indexes, and pgvector HNSW cosine index.

`database/ddl/financial_market_schema.sql, .gitignore`

Commit 2: feat(data): implement market data generator and synthetic crash seeder — AMAN RAJ • Sprint 1

Python script populating sectors, companies, stocks, and 12-month OHLCV data with injected crash anomalies.

`scripts/requirements.txt, scripts/seed_market_data.py`

Commit 3: feat(vector): add news embeddings generator using 384d sentence-transformers — AMRIT RAJ ANMOL • Sprint 1

Generates financial news headlines, computes 384-d vectors via MiniLM-L6-v2, and populates news_article and event_news.

`scripts/seed_news_embeddings.py`

Commit 4: feat(db): add market overview views and analytical rollups — AMAN RAJ • Sprint 1

Creates SQL views vw_stock_daily_stats (SMA & returns) and vw_market_overview (sector-level aggregation).

`database/advanced/03_views.sql`

Commit 5: feat(db): implement automated alert trigger for detected anomalies — AMRIT RAJ ANMOL • Sprint 1

Adds PostgreSQL AFTER INSERT trigger on anomaly table to generate user alerts for watched stocks.

`database/advanced/02_triggers.sql`

Commit 6: feat(db): implement stored procedure for multi-indicator anomaly detection — AMAN RAJ • Sprint 1

Defines sp_detect_market_anomalies using rolling standard deviation and volume surge thresholds.

`database/advanced/01_procedures_functions.sql`

Commit 7: feat(db): add semantic vector similarity search stored function — AMRIT RAJ ANMOL • Sprint 1

Defines fn_match_news_semantic utilizing HNSW cosine distance operator (<=>) against news_article.

`database/advanced/01_procedures_functions.sql`

Commit 8: docs(db): add rubric evaluation queries covering joins, aggregates, and acid transactions — AMAN RAJ • Sprint 1

Prepares SQL test scripts demonstrating GROUP BY, HAVING, subqueries, and multi-table ACID transaction blocks.

`database/advanced/04_evaluation_queries.sql`

Sprint 2: Next.js Setup, Connection Pooling & Backend API Routes

Commit 9: chore(app): setup Next.js project structure, tailwindcss, and postgres client pool — AMAN RAJ • Sprint 2

Initializes Next.js app, configures Tailwind CSS, environment variables, and creates pg.Pool connection module.

`package.json, tsconfig.json, tailwind.config.ts, src/lib/db.ts, .env.example`

Commit 10: feat(auth): implement password hashing, jwt auth helpers and auth api routes — AMRIT RAJ ANMOL • Sprint 2

Adds bcrypt password encryption, JWT generation, and endpoints for user login, registration, and session info.

`src/lib/auth.ts, src/app/api/auth/login/route.ts, src/app/api/auth/register/route.ts`

Commit 11: feat(api): create stocks and historical ohlcv data endpoints — AMAN RAJ • Sprint 2

Builds /api/stocks list route and /api/stocks/[ticker] endpoint returning candlestick history and company profile.

`src/app/api/stocks/route.ts, src/app/api/stocks/[ticker]/route.ts`

Commit 12: feat(api): implement semantic news search and watchlist crud endpoints — AMRIT RAJ ANMOL • Sprint 2

Builds /api/semantic-search connecting text query to vector search, and /api/watchlist management endpoints.

`src/lib/embeddings.ts, src/app/api/semantic-search/route.ts, src/app/api/watchlist/route.ts`

Commit 13: feat(api): add anomaly querying and trigger execution route handlers — AMAN RAJ • Sprint 2

Builds /api/anomalies endpoint for filtered anomaly logs and manual invocation of sp_detect_market_anomalies.

`src/app/api/anomalies/route.ts, src/app/api/anomalies/[id]/route.ts`

Commit 14: feat(api): implement user alert retrieval and read-status toggle endpoints — AMRIT RAJ ANMOL • Sprint 2

Creates /api/alerts route to fetch unread anomaly notifications and PATCH route to update read status.

`src/app/api/alerts/route.ts`

5. STEP-BY-STEP GITHUB COMMIT ROADMAP (PART 2: COMMITS 15 TO 24)

Sprint 3: Next.js Frontend Dashboards & Visualizations

Commit 15: feat(ui): build global navigation bar, auth modals, and notification bell — AMRIT RAJ ANMOL • Sprint 3

Creates responsive header navbar, user profile badge, unread alert counter bell, and login/register UI.

`src/components/Navbar.tsx, src/components/AlertDropdown.tsx, src/app/login/page.tsx`

Commit 16: feat(ui): develop market overview dashboard with sector metrics and summary cards — AMAN RAJ • Sprint 3

Main landing page showcasing market breadth, active anomalies counter, top movers, and sector turnover cards.

`src/app/page.tsx, src/components/MarketOverviewCard.tsx`

Commit 17: feat(ui): add interactive stock candlestick/line chart and anomaly flag overlay — AMAN RAJ • Sprint 3

Builds Recharts time-series chart with zoom/filter and visual flags highlighting days with detected anomalies.

`src/components/StockChart.tsx, src/app/stocks/[ticker]/page.tsx`

Commit 18: feat(ui): create semantic news search interface with similarity score cards — AMRIT RAJ ANMOL • Sprint 3

UI allowing natural language news search, displaying cosine similarity percentages and linked crash events.

`src/app/semantic-search/page.tsx, src/components/SemanticSearchBox.tsx`

Commit 19: feat(ui): create anomaly explorer table with severity filtering and indicator details — AMAN RAJ • Sprint 3

Interactive anomaly table with multi-column sorting, severity badge filters, and modal revealing Z-score deviations.

`src/app/anomalies/page.tsx, src/components/AnomalyTable.tsx`

Commit 20: feat(ui): build user watchlist management page and preference settings — AMRIT RAJ ANMOL • Sprint 3

Allows users to add/remove stocks from personal watchlists and configure anomaly alert sensitivity thresholds.

`src/app/watchlist/page.tsx, src/components/WatchlistManager.tsx`

Sprint 4: Verification, Evaluation Queries & Documentation

Commit 21: docs: generate comprehensive system architecture and data dictionary — AMAN RAJ • Sprint 4

Documents 18 tables, column data types, business constraints, and system data-flow diagrams.

`docs/SYSTEM_ARCHITECTURE.md, docs/DATA_DICTIONARY.md`

Commit 22: docs: write project setup guide, vector db justification, and testing walkthrough — AMRIT RAJ ANMOL • Sprint 4

Documents local setup instructions, pgvector technical rationale, and step-by-step reproduction guide.

`README.md, docs/ADVANCED_DATABASE_FEATURES.md`

Commit 23: test: verify end-to-end anomaly detection pipeline and fix edge-case queries — AMAN RAJ • Sprint 4

End-to-end validation of market data generation, stored procedure execution, and query execution times.

`database/advanced/04_evaluation_queries.sql, src/lib/db.ts`

Commit 24: test: validate vector cosine distance search accuracy and alert trigger responses — AMRIT RAJ ANMOL • Sprint 4

Verifies HNSW index latency (<10ms), validates trigger event propagation to user alerts, and polishes UI.

`src/app/api/semantic-search/route.ts, README.md`

6. VIVA / EVALUATION CHECKLIST (HOW TO DEFEND YOUR WORK)

Use this guide during the final project viva to demonstrate clear individual ownership of database and application components.

Evaluation Area	Aman Raj (PES2UG24CS053) Explains	Amrit Raj Anmol (PES2UG24CS056) Explains
Relational Modeling & Constraints	• Foreign keys and composite PK on `trading_day(exchange_id, trade_date)`. • Multi-column CHECK constraints ensuring non-negative prices, volumes, and `high >= low`. • Unique constraint on `(stock_id, trade_date)` preventing duplicate market quotes.	• 18-table normalized database architecture across 4 distinct functional sub-domains. • CASCADE delete behaviors on user watchlists, watchlist items, and alerts. • Unique constraints on ISIN numbers and exchange-ticker combinations.
Vector DB & Semantic Search	• Integration of vector search results into equity crash timelines. • Correlating price drop timestamps with detected news article publication dates.	• Justification for selecting `pgvector` over external vector DBs (zero dual-database sync overhead). • Dense 384-dimensional vector embeddings generated via MiniLM. • HNSW indexing structure using cosine distance (`vector_cosine_ops`).
Advanced DB Concepts (Min. 4)	• **Stored Procedure:** `sp_detect_market_anomalies` calculating multi-day rolling Z-scores. • **Views:** `vw_stock_daily_stats` and `vw_market_overview`. • **ACID Transactions:** Market data batch ingestion with atomic rollback.	• **Trigger:** `trg_on_anomaly_create_alert` automatically populating user alert feeds. • **Stored Function:** `fn_match_news_semantic` executing cosine similarity queries. • **Indexing Strategy:** Combined B-Tree composite indexes + Vector HNSW.
Queries & DML Operations	• Complex queries with 4-way JOINs, GROUP BY sector, and HAVING clauses filtering anomalies. • Correlated subqueries detecting equities moving inversely to their sector benchmark.	• DML operations managing watchlist additions, preference toggles, and alert read status updates. • Full-text search and vector similarity ranking queries.
Application & Security	• Next.js server connection pooling (`pg.Pool`) with resource reuse. • Zero credential hardcoding: strict environment variable isolation (`.env.local`). • Interactive stock candlestick & volume charts using Recharts.	• Role-Based Access Control (Admin vs. Analyst) protecting sensitive endpoints. • Cryptographic password hashing using bcryptjs. • Real-time alert polling & notification state synchronization in UI.

7. RECOMMENDED EXECUTION ORDER FOR THE TEAM

1. **Step 1 (Immediate):** Create the initial Git commit for the repository containing the existing `financial_market_schema.sql` (authored by Amrit).
2. **Step 2 (Day 1–2):** Aman builds and runs `seed_market_data.py`; Amrit builds and runs `seed_news_embeddings.py` to populate the database with realistic data.
3. **Step 3 (Day 3–4):** Implement the advanced SQL scripts (`01_procedures_functions.sql`, `02_triggers.sql`, `03_views.sql`).
4. **Step 4 (Day 5–7):** Scaffold the Next.js frontend and build the backend API route handlers.
5. **Step 5 (Day 8–10):** Build the UI pages (Dashboard, Charts, Semantic Search, Watchlists, Alerts).
6. **Step 6 (Day 11):** Execute evaluation queries, capture application screenshots, and finalize project documentation.

— PES University • Department of Computer Science & Engineering • Sem 5 DBMS Experiential Learning —